Features

În ordinea în care se construiesc · bifezi când merge pe EC2 · Sebi

Starea la `ab630df`: verde = merge · cărămiziu = scris, dar nu merge · gri sau fără stare = de scris

1 Conturi

vizitator · orice utilizator · administrator

- ☐ **F1.1** Vizitatorul își face cont de proprietar: nume, prenume, vârstă, email, parolă. — `register`
  scris · oricine își poate alege singur `userType: ADMIN`
- ☐ **F1.2** Utilizatorul se autentifică și primește un token. — `login`
  merge
- ☐ **F1.3** Utilizatorul autentificat își vede contul și permisiunile. — `getCurrentUser`
- ☐ **F1.4** Administratorul creează un cont `OPERATOR` sau `USER`. — `addUser`
  scris · 403 pentru toată lumea (`ADMIN` nu e permisiune); nu salvează parola și tipul

Reguli

- Email valid și unic (409); parolă de minim 8 caractere; vârstă ≥ 18. Prin înregistrare se face doar `USER`. Contul `ADMIN` îl creează aplicația la pornire, din `ADMIN_EMAIL` / `ADMIN_PASSWORD`.
- Login greșit → 401, același mesaj pentru email inexistent și parolă greșită. Parola nu apare în niciun răspuns.
- Permisiunile vin din tip: fiecare string din `hasAuthority(...)` există în `UserPermissions` și îl primește cine trebuie.

2 Case

administrator · operator · proprietar

- ☐ **F2.1** Administratorul adaugă o casă: nume, putere PV, limite de import și export, proprietar. Bateria ei se creează odată cu ea. — `createHouse`
  scris · 403 pentru toată lumea: cere `house:manage`, token-ul are `HOUSE_MANAGE` · bateria nu se creează nicăieri
- ☐ **F2.2** Administratorul modifică o casă: tot (`PUT`) sau doar câmpurile trimise (`PATCH`). — `updateHouse` · `patchHouse`
  scris · același 403
- ☐ **F2.3** Administratorul șterge o casă fără istoric. — `deleteHouse`
  scris · același 403
- ☐ **F2.4** Proprietarul își vede casa; administratorul și operatorul văd toate casele. — `getAllHouses` · `getHouseById`
  scris · același 403 · apoi: lista dă toate casele oricui, străinul primește 500 (excepție fără handler)

Reguli

- Nume obligatoriu și unic (400 / 409); puterile > 0 (400); proprietar inexistent → 404. Casa nouă are bateria implicită: 10 kWh, SoC 50%, 3 kW / 3 kW, randament 95%.
- Casa cu telemetrie, comenzi sau tranzacții nu se șterge (409); o scoți din simulare cu `enabled: false`.
- Casa altcuiva → 403 cu corp `ApiErrorResponse` complet; casă inexistentă → 404, la fel, nu cu corp gol.

3 Bateria casei

proprietar · operator · administrator

- ☐ **F3.1** Proprietarul își vede bateria: SoC, puteri maxime, randament. — `getBattery`
  merge pentru proprietar și admin · străinul primește 500 (excepție fără handler)
- ☐ **F3.2** Proprietarul schimbă puterile maxime de încărcare și de descărcare. — `updateConfiguration`
  scris · răspunde 200 și nu salvează (`readOnly = true`) · nu verifică al cui e bateria
- ☐ **F3.3** Proprietarul trimite o comandă: `CHARGE`, `DISCHARGE` sau `IDLE`, cu puterea în kW. — `sendCommand`
  scris · `targetSoc` intră în motor ca kW · proprietarul n-are `BATTERY_COMMAND` → 403
- ☐ **F3.4** Proprietarul vede istoricul comenzilor casei, cu rezultatul fiecăreia. — `getCommands`
  scris · merge doar pentru admin: proprietarul n-are `BATTERY_COMMAND`

Reguli

- Proprietarul doar la casa lui; `ADMIN` și `OPERATOR` la orice casă; străinul → 403.
- O comandă acționează un tick (1 h virtuală). Peste puterea maximă sau cu SoC-ul ieșit din 10–90% → se salvează `REJECTED`, cu motivul (`BATTERY_POWER_LIMIT` / `BATTERY_SOC_LIMIT`), iar bateria nu se schimbă. Orice altă eroare nu e `REJECTED`.
- Comanda salvată are SoC-ul de după: `CHARGE` 2 kW pe SoC 50, 10 kWh, 95% → `SUCCESS`, SoC 69.

4 Simulare și telemetrie

administrator · proprietar · operator

- ☐ **F4.1** La prima pornire cu profilul `demo` există comunitatea: 10 case cu baterii și 10 proprietari. — la pornire
- ☐ **F4.2** Administratorul rulează un tick: fiecare casă activă primește producție PV, consum, bilanț și un rând de telemetrie. — `runSimulationTick`
  de scris · `EnergyEngine` e scris și testat, dar nu-l cheamă nimeni
- ☐ **F4.3** Proprietarul vede ultimele 50 de măsurători ale casei, cele mai noi primele. — `getHouseTelemetry`
  de scris · entitatea există, `TelemetryRepository` e comentat
- ☐ **F4.4** Operatorul vede starea comunității: PV total, consum total, rețea, SoC mediu. — `getCommunityState`

Reguli

- Aceeași sămânță (`solarsync.simulation.seed`) și aceeași oră → aceleași valori. Noaptea PV = 0; la prânz, aproape de `pvPeakPowerKw` .
- Un tick = o oră virtuală (`tick-seconds` × `real-to-virtual-ratio` = 60 × 60 s). Până la agent, bateriile stau `IDLE` în tick.
- Tick doar `ADMIN` (403); telemetria casei altcuiva → 403. A doua pornire nu mai creează nimic.

5 Financiar

proprietar · operator

- ☐ **F5.1** La fiecare tick, importul casei se înregistrează ca `BUY` , exportul ca `SELL` , la prețul tick-ului. — în tick
  scris · `recordTransaction` n-are niciun apelant
- ☐ **F5.2** Proprietarul vede tranzacțiile casei lui. — `getHouseFinancialSummary`
  scris · cale fără `/api/v1` · `FINANCIAL_VIEW` nu există → 403 cu token real
- ☐ **F5.3** Operatorul vede tranzacțiile întregii comunități. — `getCommunityFinancialSummary`
  scris · la fel, cu `COMMUNITY_VIEW` · testul trece doar cu `@WithMockUser`

Reguli

- Valoarea = kWh × preț, pe `BigDecimal` . Prețul vine dintr-un generator determinist pe oră, ca PV-ul.
- Proprietarul vede doar casa lui (403). Cele mai noi primele.

6 Rețeaua

operator · administrator

- ☐ **F6.1** La fiecare tick se calculează încărcarea transformatorului comun și se salvează starea. — în tick
  de scris · `GridProperties` există, nefolosit
- ☐ **F6.2** Operatorul vede starea rețelei după ultimul tick. — `getGridState`

Reguli

- Transformator 50 kW, marjă 10% → limită 45 kW. Import și export separat: ≤ 45 `OK` , până la 50 `WARNING` , peste `CRITICAL` , cu kW-ii în plus.

7 Agentul

operator · administrator

- ☐ **F7.1** Operatorul pornește o rulare: agentul propune câte o comandă pe casă, cu motivul, și o trece prin validator. — `runAgent`
  de scris · `BatteryCommandValidator` există
- ☐ **F7.2** Operatorul vede rulările și deciziile fiecăreia, valide și respinse. — `getAgentRuns` · `getAgentRun`
- ☐ **F7.3** Operatorul aprobă o rulare: deciziile valide devin comenzi reale. Sau o respinge. — `approveAgentRun` · `rejectAgentRun`

Reguli

- Protecția transformatorului bate optimizarea: import `CRITICAL` → descarcă bateriile cu SoC peste 30%; export `CRITICAL` sau surplus PV → încarcă.
- În `ADVISORY` deciziile se salvează, nu se execută: după rulare SoC-urile sunt neschimbate. Decizia respinsă nu se execută niciodată.
- Aprobarea folosește aceeași cale ca `sendCommand` ; o rulare se hotărăște o singură dată (409).

8 Simularea pornește singură · opțional

administrator

- ☐ **F8.1** Tick-ul rulează singur la `tick-seconds` ; agentul la `planning-interval-seconds` . — automat
- ☐ **F8.2** Administratorul vede starea simulării, o pornește și o pune pe pauză. — `getSimulationState` · `controlSimulation`